

Core SQL Interview Questions With Answers

Part 1: Basics

1. What is SQL?

- SQL stands for *Structured Query Language*.
- It is used to read and manage data in relational databases.
- Common systems: MySQL, PostgreSQL, SQL Server, Oracle.

2. What is an RDBMS?

- RDBMS stands for *Relational Database Management System*.
- Data is stored in tables with rows and columns.
- Keys are used to link tables.
- Example: `customer_id` links a `Customer` table to an `Orders` table.

3. What is a table?

- A table is structured storage for data.
- Rows are individual records.
- Columns are attributes or fields.
- Example: one row can represent one customer.

4. What is a primary key?

- Uniquely identifies each row in a table.
- Cannot be `NULL`.
- Cannot contain duplicate values.
- Example: `user_id` in a `users` table.

5. What is a foreign key?

- A foreign key links one table to another.
- It refers to a primary key in another table.
- It can contain duplicate values.
- Example: `user_id` in an `orders` table.

6. Difference between primary key and foreign key

- Primary key ensures each row is unique.
- Foreign key enforces relationships between tables.
- One table typically has one primary key.
- One table can have multiple foreign keys.

7. What is `NULL`?

- Represents a missing or unknown value.
- It is not equal to zero or an empty string.
- Use `IS NULL` or `IS NOT NULL` to check it.

8. What are constraints?

- Constraints are rules applied to columns.
- They help maintain data quality.
- Common constraints include:
 - `NOT NULL`
 - `UNIQUE`
 - `PRIMARY KEY`
 - `FOREIGN KEY`
 - `CHECK`

9. What are data types?

- Data types define the kind of values stored in a column.
- Common types:
 - `INT` for integers
 - `VARCHAR` for text
 - `DATE` for dates
 - `FLOAT` / `DECIMAL` for decimals

10. Interview tip:

- Explain with a small example.
- Speak logic before syntax.
- Keep answers concise and direct.

Part 2: Core Queries

11. What is `SELECT`?

- Retrieves specific data from tables.
- Use `SELECT *` for all columns or list columns explicitly.
- Example:

```
SELECT name, age  
FROM users;
```

12. What does `WHERE` do?

- Filters rows based on conditions.
- It is applied after the `FROM` clause.
- Example:

```
SELECT *  
FROM users  
WHERE age > 25;
```

13. What is **ORDER BY**?

- Sorts the result set by one or more columns.
- Default order is **ASC**; use **DESC** for descending.
- Example:

```
SELECT *  
FROM users  
ORDER BY age DESC;
```

14. What is **GROUP BY**?

- Groups rows that share the same values.
- It is used with aggregate functions like **COUNT** and **SUM**.
- Example:

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

15. Difference between **WHERE** and **HAVING**

- **WHERE** filters rows before grouping.
- **HAVING** filters groups after **GROUP BY**.
- **WHERE** cannot use aggregates; **HAVING** can.

16. What is a **JOIN**?

- Combines rows from two or more tables.
- It uses related columns to match rows.
- **INNER JOIN** returns only matching rows.

17. Types of joins

- **INNER JOIN**: matching rows only.
- **LEFT JOIN**: all rows from left table plus matches from right.
- **RIGHT JOIN**: all rows from right table plus matches from left.
- **FULL OUTER JOIN**: all rows from both tables.

18. What are aggregate functions?

- Perform calculations on groups of rows.
- Common functions: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**.
- Example:

```
SELECT AVG(salary)  
FROM employees;
```

19. What is a subquery?

- A subquery is a query within another query.
- It can appear in **SELECT**, **WHERE**, or **FROM**.
- Example:

```
SELECT *  
FROM users  
WHERE age > (  
    SELECT AVG(age)  
    FROM users  
);
```

20. Interview tip:

- Explain the SQL execution order: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**.
- Use simple examples with a few rows.
- Practice on sample data sets like **employees** and **orders**.

Part 3: Advanced Concepts

21. What is a CTE (Common Table Expression)?

- A CTE is a named temporary result set defined with **WITH**.
- It improves readability and allows breaking queries into steps.
- Example:

```
WITH high_salaries AS (  
    SELECT *  
    FROM employees  
    WHERE salary > 80000  
)  
SELECT name, salary  
FROM high_salaries  
WHERE dept_id = 101;
```

22. What are window functions?

- They compute values across a window of rows without collapsing rows.
- Use **OVER(...)** after an aggregate or ranking function.
- Example:

```
SELECT customer_id, order_date, amount,  
    SUM(amount) OVER (  
        PARTITION BY customer_id
```

```
ORDER BY order_date
) AS running_total
FROM orders;
```

23. Difference between **GROUP BY** and window functions

- **GROUP BY** collapses rows into aggregated groups.
- Window functions keep all rows and add computed values.
- Use **GROUP BY** for summary results, window functions for row-level analytics.

24. What are ranking functions?

- **ROW_NUMBER()**: assigns unique sequential numbers.
- **RANK()**: leaves gaps for ties.
- **DENSE_RANK()**: assigns ranks without gaps.
- Example:

```
WITH ranked_orders AS (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date DESC
         ) AS rn
  FROM orders
)
SELECT *
FROM ranked_orders
WHERE rn = 1;
```

25. What is the difference between **UNION** and **UNION ALL**?

- **UNION** removes duplicate rows.
- **UNION ALL** preserves duplicates and is faster.
- Both require the same number of columns and compatible data types.

26. Difference between **DISTINCT** and **GROUP BY**

- **DISTINCT** removes duplicate rows from the final result.
- **GROUP BY** groups rows and allows aggregation.
- Example:

```
SELECT dept_id, COUNT(DISTINCT employee_id)
FROM employees
GROUP BY dept_id;
```

27. How do you handle **NULLs**?

- Use **IS NULL** / **IS NOT NULL** for comparisons.

- Use `COALESCE(expr, default)` to replace nulls.
- Example:

```
SELECT name,  
       COALESCE(email, 'No email') AS email  
FROM customers;
```

28. What is an index and why use it?

- An index speeds up data retrieval.
- It helps `WHERE`, `JOIN`, and `ORDER BY` queries.
- But indexes can slow down `INSERT`, `UPDATE`, and `DELETE`.

29. How do you optimize a slow SQL query?

- Add indexes on frequently filtered columns.
- Avoid `SELECT *`; select only needed columns.
- Prefer `INNER JOIN` over subqueries when appropriate.
- Use pagination with `LIMIT`.
- Avoid wrapping indexed columns in functions.

30. What is the basic SQL execution order?

- `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT` / `OFFSET`.
- Use this to explain filter placement.

Quick Interview Advice

- Use real examples to support your answers.
- Explain the reasoning before writing syntax.
- Mention indexes and execution order in performance questions.
- Keep the response short, clear, and practical.